SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	J. LOKESH	**Reg. No.:**	192311216
Date:	23/08/2026	**Duration:**	60 minutes

Code of Conduct

I J. LOKESH 192311216(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: J.LOKESH

Annexure A – CI/CD Pipeline Design Exercise

Smart Tax Calculation and Financial Planning Management System

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollbac k details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/docum entation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	**Mark**

Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Introduction and Project Overview

The Smart Tax Calculation and Financial Planning Management System is a web-based financial technology application designed to help individual users and small businesses compute income tax liabilities, plan investments, track expenses, and generate personalized financial recommendations. The system integrates real-time tax slab data, investment rules, and user financial profiles to produce accurate tax estimates and actionable financial planning insights. As the system handles sensitive financial and personally identifiable information (PII), frequent changes to tax regulations, and a growing user base, it requires a disciplined software delivery process. A well-designed Continuous Integration and Continuous Deployment (CI/CD) pipeline ensures that every code change is automatically built, rigorously tested, statically and dynamically analyzed for security and quality, and safely deployed through progressively stricter environments before reaching end users. This document presents the complete CI/CD pipeline design for the system, covering requirement analysis, pipeline architecture, tool selection, automated testing strategy, deployment strategy, and security and rollback mechanisms.

1.1 Purpose of this Document

This document serves as the design blueprint for implementing an automated software delivery pipeline for the Smart Tax Calculation and Financial Planning Management System. It is intended for the development team, DevOps engineers, quality assurance team, and project stakeholders who will implement, operate, and maintain the pipeline.

1.2 Scope

- Analysis of CI/CD requirements specific to a financial/tax-domain application.
- End-to-end pipeline architecture from source code commit to production deployment.

- Selection and justification of tools and technologies for each pipeline stage.
- Automated testing strategy covering functional correctness and financial-calculation accuracy.
- Deployment strategy across development, staging, and production environments.
- Security controls, quality gates, notification mechanisms, and rollback procedures.

2. Project Requirements and CI/CD Needs Analysis

Before designing the pipeline, it is essential to understand the functional and non-functional characteristics of the Smart Tax Calculation and Financial Planning Management System that directly influence CI/CD design decisions.

2.1 System Characteristics

- Multi-module architecture: tax calculation engine, financial planning/recommendation engine, user profile and authentication module, reporting and dashboard module, and a REST API layer.
- Frequent business-rule updates: annual and mid-year changes to tax slabs, deduction limits, and investment regulations require rapid, low-risk releases.
- High accuracy requirement: incorrect tax computation has direct financial and legal consequences for users, demanding rigorous automated regression testing.
- Sensitive data handling: PII, income details, and bank/investment information require encryption, access control, and compliance-oriented security scanning.
- Variable load patterns: usage spikes sharply around tax filing deadlines, requiring scalable, container-based deployment.

2.2 Identified CI/CD Requirements

Requirement	Description	Pipeline Implication
Fast, reliable integration	Multiple developers commit to shared modules daily	Automated build + unit test on every commit/PR
Calculation accuracy assurance	Tax and planning logic must match regulatory rules	Dedicated regression test suite with golden datasets

Requirement	Description	Pipeline Implication
Security & compliance	Sensitive financial data, regulatory exposure	SAST/DAST, dependency scanning, secrets scanning
Safe, frequent releases	Tax rule updates need quick, low-risk rollout	Staged deployment with canary/blue-green strategy
High availability	Peak load during filing season	Container orchestration with auto-scaling
Traceability & auditability	Financial application, audit requirements	Versioned artifacts, immutable build logs, approvals

3. CI/CD Pipeline Architecture and Workflow

The proposed pipeline follows a linear, gated-stage model where each stage must pass defined quality criteria before the build proceeds to the next stage. Failures at any stage immediately halt the pipeline, notify the responsible team, and prevent flawed code from reaching later environments. Figure 1 illustrates the end-to-end workflow from source code commit to production deployment, including the environment progression and the automated rollback/feedback loop.

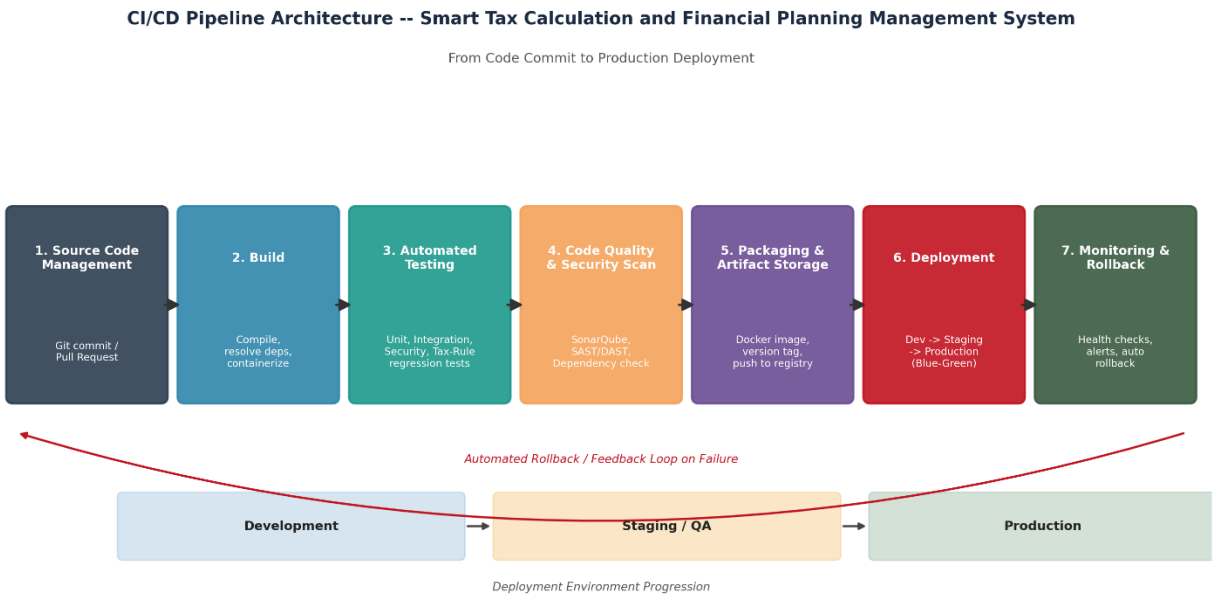

Figure 1: CI/CD Pipeline Architecture for the Smart Tax Calculation and Financial Planning Management System

3.1 Stage-by-Stage Workflow Description

Stage 1: Source Code Management

Developers work on feature branches (feature/, bugfix/, hotfix/) branched from a protected 'develop' branch, following a GitFlow-inspired branching model. Every push to a feature branch and every pull request targeting 'develop' or 'main' automatically triggers the pipeline via webhooks. Pull requests require at least one peer code review approval and a passing pipeline run before merging.

Stage 2: Build

The build stage resolves dependencies, compiles the backend services (tax engine and planning engine), builds the frontend assets, and produces containerized artifacts using Docker. Build caching is used to minimize execution time. A failed build immediately stops the pipeline and notifies the committer.

Stage 3: Automated Testing

Automated tests run in parallel where possible: unit tests validate individual functions such as tax bracket calculations and deduction logic; integration tests validate interactions between the tax engine, planning engine, and database; and a dedicated tax-rule regression suite compares calculation outputs against verified golden datasets representing real tax scenarios across multiple income brackets and financial years.

Stage 4: Code Quality and Security Analysis

Static code analysis measures maintainability, code smells, duplication, and cyclomatic complexity. Static Application Security Testing (SAST) scans source code for vulnerabilities such as injection flaws and insecure cryptographic usage. Software Composition Analysis checks third-party dependencies for known CVEs. A quality gate enforces minimum thresholds (for example, no critical vulnerabilities, and code coverage above a defined percentage) before the pipeline can proceed.

Stage 5: Packaging and Artifact Storage

Once a build passes all quality gates, it is packaged into a versioned, immutable Docker image tagged with the commit hash and semantic version, then pushed to a private container registry. Build metadata, test reports, and security scan results are archived for audit and traceability.

Stage 6: Deployment

Approved artifacts are automatically deployed to the Development environment for continuous verification, then promoted to Staging/QA for manual and exploratory testing, and finally to Production using a blue-green deployment strategy that minimizes downtime and risk. Promotion to Production requires an explicit manual approval gate from a release manager.

Stage 7: Monitoring and Rollback

Post-deployment, automated health checks and Dynamic Application Security Testing (DAST) scans validate the running application. Application performance monitoring and error-rate tracking continuously observe production behavior. If health checks fail or error rates exceed defined thresholds, the pipeline automatically triggers a rollback to the last known-good version and alerts the DevOps team.

4. Tools and Technology Selection

The following tools were selected based on industry adoption, integration compatibility, support for containerized microservice architectures, and suitability for a security-sensitive financial application.

Pipeline Stage	Selected Tool(s)	Justification
Version Control & SCM	Git + GitHub (branch protection, PR reviews)	Industry-standard distributed VCS; native webhook support for pipeline triggers; enforced review policies aid audit compliance.
CI/CD Orchestration	GitHub Actions / Jenkins	Native integration with GitHub; declarative YAML pipelines; large plugin ecosystem for financial and security tooling.

Pipeline Stage	Selected Tool(s)	Justification
Build & Dependency Mgmt	Maven/Gradle (Java backend), npm (frontend), Docker	Mature, widely supported build tools; Docker ensures consistent, reproducible environments across all stages.
Unit & Integration Testing	JUnit5/Mockito (backend), Jest (frontend), Postman/Newman (API)	Comprehensive coverage of business logic, UI components, and REST API contracts with automatable reporting.
Code Quality Analysis	SonarQube	Detects code smells, duplication, and complexity; enforces configurable quality gates before promotion.
Security Scanning (SAST)	Snyk Code / Semgrep	Identifies insecure coding patterns and injection risks specific to financial data handling.
Dependency/SCA Scanning	OWASP Dependency-Check / Snyk	Flags known CVEs in third-party libraries used by the tax and planning engines.
Dynamic Security Testing (DAST)	OWASP ZAP	Simulates external attacks against the running application to catch runtime vulnerabilities pre- and post-deployment.
Artifact Repository	Docker Hub (private) / Nexus Repository	Secure, versioned storage of build artifacts with access control and retention policies.

Pipeline Stage	Selected Tool(s)	Justification
Container Orchestration	Kubernetes	Enables auto-scaling for seasonal tax-filing load spikes, self-healing, and blue-green/canary rollout strategies.
Infrastructure as Code	Terraform	Ensures environments (Dev/Staging/Prod) are reproducible, version-controlled, and consistently provisioned.
Monitoring & Logging	Prometheus + Grafana, ELK Stack	Real-time metrics, dashboards, and centralized log aggregation for rapid incident detection.
Notifications	Slack / Microsoft Teams + Email webhooks	Immediate alerts to developers and DevOps on build failures, security findings, and deployment status.
Secrets Management	HashiCorp Vault	Secure storage and rotation of API keys, database credentials, and encryption keys used by the pipeline.

5. Automated Testing Strategy

Given that incorrect tax computation carries real financial consequences, testing strategy for this system goes beyond conventional software testing to include domain-specific validation layers.

5.1 Testing Levels

1. Unit Testing: Validates individual functions such as slab-wise tax computation, deduction eligibility checks, and interest/return calculations, executed on every commit with a minimum coverage threshold of 85%.

2. Integration Testing: Validates interactions between the tax engine, financial planning engine, user profile service, and the database layer, executed automatically after successful unit tests.
3. Tax-Rule Regression Testing: A specialized suite that runs computed results against a curated 'golden dataset' of verified tax scenarios (multiple income levels, deduction combinations, and financial years) to guarantee that new code does not silently break calculation accuracy.
4. API Contract Testing: Automated Postman/Newman collections validate that REST API request/response contracts remain backward compatible for the mobile and web front ends.
5. Security Testing: SAST scans run on every commit; DAST scans run against the staging environment before production promotion; dependency scans run daily and on every build.
6. Performance/Load Testing: Executed in the staging environment using JMeter to simulate peak tax-filing-season traffic before each major release.
7. User Acceptance Testing (UAT): Manual exploratory and scenario-based testing performed by the QA team in the staging environment prior to production approval.

5.2 Quality Gates

The pipeline enforces the following automated quality gates. A build cannot progress to the next stage unless all applicable gates pass:

- Unit and integration test pass rate: 100% (zero failing tests).
- Code coverage: minimum 85% for the tax calculation and financial planning modules.
- Tax-rule regression suite: 100% match against golden dataset outputs.
- SonarQube quality gate: no blocker or critical issues.
- Security scan: zero critical or high-severity vulnerabilities permitted in production builds.

6. Deployment Strategy

Deployments progress through three isolated environments, each with increasing stability and access restrictions, to progressively reduce risk before changes reach real users.

Environment	Purpose	Deployment Trigger	Data
Development	Continuous integration verification; developer and automated testing	Automatic on every successful merge to 'develop'	Synthetic/anonymized test data
Staging / QA	Manual QA, UAT, performance and DAST testing	Automatic on every successful merge to 'main' after Dev verification	Masked production-like data
Production	Live system serving end users	Manual approval by release manager after staging sign-off	Live, encrypted user financial data

6.1 Deployment Technique: Blue-Green Deployment

Production releases use a blue-green deployment model: the new version ('green') is deployed alongside the currently live version ('blue') within the Kubernetes cluster. Automated smoke tests and health checks validate the green environment before traffic is gradually switched over using the load balancer. The blue environment is kept running for a defined observation window, allowing an immediate traffic switch-back if issues are detected, without redeploying.

6.2 Release Cadence

- Feature releases: bi-weekly, following full pipeline validation.
- Tax-rule hotfixes: expedited pipeline path with mandatory regression suite execution, deployable within hours when regulatory deadlines require it.
- Security patches: prioritized and fast-tracked through the standard pipeline gates.

7. Security, Quality Checks, Notifications, and Rollback Mechanisms

7.1 Security Controls

- Static Application Security Testing (SAST) integrated into every pipeline run to catch insecure code before merge.
- Software Composition Analysis (SCA) to detect vulnerable third-party dependencies used by the tax and planning engines.

- Dynamic Application Security Testing (DAST) executed against the staging environment prior to production promotion.
- Secrets management via HashiCorp Vault; no credentials or API keys are stored in source control or pipeline scripts.
- Encryption of sensitive financial data in transit (TLS 1.2+) and at rest (AES-256).
- Role-based access control (RBAC) restricting who can approve production deployments.

7.2 Quality Checks

- Mandatory peer code review and passing pipeline status before any merge is permitted.
- SonarQube quality gate enforcing maintainability, reliability, and security ratings.
- Automated linting and formatting checks for consistency across the codebase.

7.3 Notification Mechanisms

The pipeline integrates with Slack and email to provide real-time visibility into build health and deployment status:

- Build/test failure notifications sent immediately to the committing developer and the team channel.
- Security scan alerts routed to the security/DevOps channel with severity classification.
- Deployment status notifications (start, success, failure, rollback) sent to stakeholders and the release manager.
- Production incident alerts triggered by monitoring thresholds, escalated to on-call engineers.

7.4 Rollback Mechanism

Rollback safety is built into every production deployment:

1. Automated health checks run immediately after traffic is shifted to the new (green) version.
2. If error rates, latency, or failed health checks exceed configured thresholds within the observation window, the pipeline automatically redirects traffic back to the previous stable (blue) version.
3. All Docker images are immutably versioned, allowing any prior release to be redeployed on demand within minutes.
4. Database migrations are designed to be backward-compatible where possible, and paired with rollback scripts to prevent data-loss during reversion.

5. A post-incident review is conducted after every rollback to identify root cause and update the regression test suite to prevent recurrence.

8. Sample Pipeline Configuration (Illustrative)

The following simplified YAML illustrates how the pipeline stages described above could be implemented using a GitHub Actions-style declarative configuration.

```
name: tax-planning-system-pipeline
on: [push, pull_request]
jobs:
  build:
    steps:
      - checkout code
      - build backend (Maven) and frontend (npm)
      - build Docker image
  test:
    needs: build
    steps:
      - run unit tests (JUnit, Jest)
      - run integration tests
      - run tax-rule regression suite
  quality-and-security:
    needs: test
    steps:
      - run SonarQube analysis
      - run SAST (Semgrep) and SCA (Snyk)
      - enforce quality gate
  package:
    needs: quality-and-security
    steps:
      - tag image with commit SHA and semver
```

```
    - push image to private registry
  deploy-dev:
    needs: package
    steps: [deploy to Development, run smoke tests]
  deploy-staging:
    needs: deploy-dev
    steps: [deploy to Staging, run DAST, run UAT, run load tests]
  deploy-production:
    needs: deploy-staging
    environment: production
    steps:
      - manual approval
      - blue-green deploy to Production
      - run automated health checks
      - rollback automatically on failure
```

9. Conclusion

The CI/CD pipeline designed for the Smart Tax Calculation and Financial Planning Management System establishes a fully automated, security-conscious, and auditable path from code commit to production deployment. By combining rigorous multi-level automated testing including a dedicated tax-rule regression suite, layered security scanning (SAST, SCA, DAST), staged deployment through Development, Staging, and Production environments, and an automated blue-green rollback mechanism, the pipeline directly addresses the accuracy, security, and reliability demands unique to a financial and tax-domain application. This design enables the development team to release features and critical tax-rule updates rapidly and safely, while maintaining the confidence and trust of end users who rely on the system for accurate financial decision-making.